

Phase –1: The Dry Run

Goal: prove your bench and your hands work on parts that cost pennies, *before* the Tap Light (\$0.5) puts an ESP32 and a soldered LED strip in front of you.

This is the step before the starter project. The Tap Light is already forgiving, but it still asks you to solder to a real LED strip and wire up a real ESP32. The Dry Run exercises **every base skill once, on disposable parts, with a clear pass/fail gate for each** — so that when something goes wrong on the Tap Light, you *know* it's the wiring and not your iron, your meter, or your technique.

Time: ~2–3 hours, can be one sitting or split. First time is slow; that's the point.

Parts you'll use (all cheap or throwaway):

- ELEGOO Electronic Component Fun Kit: the 830-point breadboard, jumper wires, resistors (the kit has 220 Ω , 330 Ω , and 1 k Ω — all usable here), red/green/yellow LEDs, and a 1N4007 diode (the kit includes five).
- A few 470 Ω resistors from your DigiKey bulk packs (the kit has no 470 Ω ; 220 Ω –1 k Ω from either source is fine).
- Scrap hookup wire (cut offcuts, any gauge).
- 2–3 pieces of heat-shrink.
- **One** piezo disc (you have 10 — this one is a practice/test unit).

Parts you will NOT touch in this phase: the WS2812B strip, the Mean Well PSU, the level shifter, the AC mains parts, and your two good Perma-Proto protoboards (reserved for the starter and main build). The ESP32 appears only in the optional Station 5, and only over USB with nothing wired to it.

Power for the lit-circuit stations (2 and 3): you need a small low-voltage source. Pick whichever you have on hand — both are correct, and both keep the expensive parts safe:

- **A 9V battery** (not in the kit — grab one; they're everywhere). Connect it to the breadboard's + and – rails (a 9V snap clip makes this clean; in a pinch, jam jumper-wire ends onto the battery terminals). Size the LED resistor for 9V: **470 Ω or 1 k Ω** . This keeps every expensive part off the bench entirely — the best fit for this phase's goal.
- **The ESP32's 5V + GND pins**, powered over USB. You own it, and for nothing more than an LED + resistor it is genuinely safe: a reversed LED simply won't light, and a dead short just trips the USB port's current limit and resets the board — no damage. Size the resistor for 5V: **220 Ω –470 Ω** . Use this if you'd rather not buy a battery.

The ELEGOO kit's power-supply module (the MB102 board that clips onto the rails) is a nicer regulated option, but it needs its *own* 6.5–12V barrel or 5V USB feed, which the kit does **not** include. Skip it for the dry run unless you already have a suitable adapter; the two options above are simpler and sufficient.

Either source is isolated and low-current — it can't damage anything even if you wire it backwards.

Station 0 — Bench + iron warmup (~20 min)

Get the bench set up per §0.1 of the design doc. Then:

- ☐ Iron set to ~330 °C / 625 °F (lead solder). Tin the tip — touch a little solder, wipe on damp sponge or brass coil.
- ☐ **PASS:** the tip is shiny silver (not dull/black), and fresh solder melts on contact within ~1 second.
- ☐ Meter self-test: set to continuity, touch the two probes together → it beeps.

If the tip won't tin (solder balls up and rolls off), the tip is oxidized or too cool — bump the temp, add flux, and wipe again before moving on. Fix this *now*; a bad tip makes every later joint miserable.

Station 1 — Multimeter literacy (~15 min, no iron)

The two meter functions this whole project depends on are **continuity** and **DC voltage** — and the single most important safety skill is **reading polarity** (reversed 5 V silently kills WS2812B strips). Drill all of it here on free parts.

- ☐ **Resistance:** measure 3 different resistors. Read their color bands first, predict the value, then confirm with the meter. **PASS:** meter reading is within tolerance ($\pm 5\%$) of the band value. (You just learned the color code *and* the meter in one move.)
- ☐ **Diode test** (the $\blacktriangleright|$ symbol): probe a diode (e.g. 1N4007) both ways. One direction shows ~0.5–0.7 V; the other shows "OL"/open. **PASS:** you can point to the cathode (the banded end) and explain why.
- ☐ **LED as a diode:** same diode-test mode on a **red, green, or yellow** LED — it glows faintly in the forward direction and shows a V_f of ~1.6–2.2 V. (Blue and white LEDs often just read "OL" on a meter's diode test — their forward voltage is higher than the meter can push — so use a low- V_f color here.) **PASS:** you can identify the long leg = anode = $+$.
- ☐ **DC volts:** set to DC V (20 V range or auto), measure your power source (the 9 V battery, or the ESP32's 5V pin to GND). **PASS:** reads the expected voltage, and you understand the sign tells you polarity (red on $+$, black on $-$, positive number).
- ☐ **Continuity:** a jumper wire beeps end-to-end; a resistor does *not* beep. **PASS:** you can tell "connected" from "not connected" by ear.

This is the exact skill that protects your \$20 strip later: before you ever apply 5 V to anything expensive, you'll meter the polarity at the destination first. Get fluent here.

Station 2 — First lit circuit on a breadboard (~20 min, no iron)

Your first real circuit. It's intentionally the **same shape** as powering the LED strip — a supply, a current-limiting resistor, and a polarity-sensitive load — just on a 10-cent LED instead of the strip.

- ☐ Bring power to the breadboard rails using your chosen source above (9 V battery, or the ESP32's 5V/GND pins).
 - ☐ Wire: **+rail** → **resistor** → **LED anode (long leg)** → **LED cathode** → **-rail**, using a **red LED** and the resistor sized for your source (470 Ω /1 k Ω at 9 V; 220 Ω –470 Ω at 5 V).
 - ☐ **PASS**: the LED lights.
 - ☐ Now deliberately **reverse the LED**. It won't light — and the resistor keeps it from being harmed. Flip it back. **PASS**: you saw that polarity matters *and* that a mistake is recoverable.
 - ☐ Bonus (optional): measure the voltage across the LED and across the resistor. Notice they add up to your supply voltage — that's a series circuit. (The piezo network on the main build is *parallel-to-ground* instead; you'll see that shape on the Tap Light.)
-

Station 3 — The soldering ladder (~40–60 min, iron)

Progressive difficulty, all on scrap and throwaway ELEGOO parts. This rehearses **every** solder move the main build needs.

- ☐ **Rung 1 — wire-to-wire (the \$0.2 warmup)**: strip 5 pieces of scrap wire, twist pairs, solder them. Inspect each. **PASS**: the last joints are shiny, wet both sides, and survive a firm tug.
- ☐ **Rung 2 — heat-shrink**: slide shrink over 2 of those joints and shrink it (lighter or hot-air, *not* a hair dryer). **PASS**: tight, centered, no scorch marks.
- ☐ **Rung 3 — component-lead joint**: solder one leg of a resistor directly to the long leg (anode) of an LED, making a free-form series pair. This is the *same* lead-to-lead joint you'll make on the strip's corner jumpers and the piezo leads, so it's the most directly relevant solder move in the whole dry run. Heat-shrink the joint, then push the two free ends into the breadboard rails to light it. **PASS**: it lights, and the joint is a clean cone — not a blob, not a ball. (*Have a scrap perfboard offcut handy? Even better — solder the pair through-hole onto it. Just don't use your two reserved Perma-Protos.*)
- ☐ **Rung 4 — rework**: take one joint from Rung 3, add flux, and remove the solder with wick/braid. Then redo the joint. **PASS**: pad and wire cleaned up, rejoined, still works.

Rung 4 matters more than it looks. You *will* make a bad joint on the real build, and being able to calmly desolder and redo it — instead of panicking on a \$20 strip — is the difference between a fixable mistake and a ruined part.

Station 4 — Piezo handling, no heat (~15 min)

Confirm a piezo is alive and see that it really converts a tap into electricity — **without soldering it** (so zero risk of heat-killing it) and without the ESP32. Two demos: the first is tactile and convincing, the second teaches what the signal looks like.

Take **one** piezo from your bag of 10. The two demos work best with clip leads (your helping-hands' alligator clips) so you're not fighting to hold probes on bare wires.

Demo A — make it flash an LED (the convincing one):

- ☐ Connect a **red LED directly across the two piezo leads** — long leg to one lead, short leg to the other. Polarity does **not** matter here: a tapped piezo puts out an AC swing, so it'll flash either way.
- ☐ Dim the room a little. Tap the brass disc — start gently and increase force until you see it. **PASS:** the LED gives a faint, brief flash on the harder taps.
- ☐ Expect it to be *dim and quick* — the piezo makes a high voltage but almost no current, so there's only enough energy for a flicker. A faint blink is a full pass; you've just watched mechanical energy become light with no battery involved.

Demo B — watch the signal on the meter (the instructive one):

- ☐ Set the meter to **DC volts**, lowest range (2 V or auto). Clip the probes to the two leads.
- ☐ **Slowly press and release** the disc — a deliberate squeeze, *not* a sharp tap. **PASS:** the number swings one way as you press and the other way as you release. A sharp tap is too fast for a cheap meter to catch cleanly and the meter's input impedance drags the reading down, so a slow press is what shows the swing — often just tenths of a volt to a couple of volts. You're confirming *life and direction of swing*, not measuring a true peak.

This is exactly why the design doc has the ESP32 read the piezo *once* per scan instead of averaging: the real signal is a fast spike that decays in milliseconds. Your meter can barely see it; the ESP32's ADC catches it easily. Both demos here just prove the transducer works before you ever risk heat on it.

- ☐ **Do not solder this piezo.** Soldering leads to a piezo is a Tap-Light step (and the doc warns: be *fast* — piezos lose sensitivity when overheated). This station is only "confirm the sensor is alive."

Station 5 — ESP32 toolchain, USB only (optional but recommended, ~30 min)

This is the **one** step that touches an expensive part — and it's the safest possible way to touch it: a USB cable and nothing else. No pins wired, no power supply, no polarity to get wrong.

- ☐ Do §0.4 of the design doc: install Arduino IDE 2.x, add ESP32 board support, install FastLED.
- ☐ Plug the ESP32-S3 into USB (confirm it's a **data** cable, not charge-only).
- ☐ Flash **File** → **Examples** → **01.Basics** → **Blink**. Board = "ESP32S3 Dev Module", **USB CDC On Boot: Enabled**.

☐ Open Serial Monitor at 115200. **PASS:** Blink uploads cleanly and the board responds. (No onboard LED reaction? Run [Examples](#) → [ESP32](#) → [ChipID](#) to confirm the chip is talking over serial.)

Why do this now: the toolchain — drivers, port selection, USB-CDC settings, download-mode button dance — is the single thing most likely to eat an evening. Debugging it with *nothing wired* means you can never wonder "is it my soldering?" Get it green here, in isolation, and the Tap Light becomes pure hardware.

Want to keep Phase –1 strictly no-expensive-parts? Skip this and make it the very first thing you do on the Tap Light instead. But doing it here de-risks the toolchain on its own, which is the whole spirit of this phase.

Exit gate → ready for the Tap Light (\$0.5)

Check all of these honestly. Each one is a tool or technique the Tap Light depends on:

- ☐ My iron heats, tins, and holds temperature.
- ☐ I can read a resistor's color bands and confirm with the meter.
- ☐ I can use continuity and DC-voltage modes, and I can read polarity off the meter.
- ☐ I built a working series circuit on a breadboard and recovered from a reversed LED.
- ☐ My wire-to-wire joints are shiny and survive a tug.
- ☐ I soldered a clean component-lead joint and heat-shrank it.
- ☐ I desoldered a joint with wick and redid it.
- ☐ I heat-shrank a joint properly.
- ☐ I confirmed a piezo is alive — it flashed an LED and/or swung the meter when pressed.
- ☐ (Optional) I flashed the ESP32 over USB and read serial output.

If all of these pass, every tool and skill the Tap Light needs is now *known-good* — so any problem on the starter project is a wiring or design issue you can isolate, not a mystery about whether your equipment even works. That's exactly the confidence this phase is meant to buy.

Total expensive parts consumed: zero (the ESP32 is only touched over USB, and survives unscathed). Everything else — a few resistors, a couple LEDs, a diode, scrap wire, two bits of heat-shrink, and one piezo out of ten — costs well under a dollar.

Phase –1 complete → proceed to §0.5, The Starter Project: Tap Light.